

CSC1110/CSC1122: Natural Language Technologies/ Foundations of NLP

Lecture 7: Word Vectors

Dr. Ellen Rushe
Assistant Professor
School of Computing
Dublin City University

L2.24 (McNulty Building)
ellen.rushe@dcu.ie

DCU

Ollscoil Chathair
Bhaile Átha Cliath
Dublin City University

The contents of these slides are based on
Chapter 6 of Jurafsky and Martin *Speech and
Language Processing* (3rd Edition)
and
[https://computational-linguistics-
class.org/slides/06-vector-semantics-part-2.pdf](https://computational-linguistics-class.org/slides/06-vector-semantics-part-2.pdf)
With special thanks to Prof. Jennifer Foster
for a wealth of past slides!

Representing words as vectors

What is a word's meaning?

"The meaning of a word is its use in the language"

Ludwig Wittgenstein (1945)

If two words have almost identical environments we say that they are synonyms.

Zellig Harris (1954)

"You shall know a word by the company it keeps"

J.R. Firth (1957)

What does ong choi mean?

- Suppose you see these sentences:
 - Ong choi is **delicious sautéed with garlic**.
 - Ong choi is superb **over rice**
 - Ong choi **leaves** with **salty** sauces
- And you've also seen these:
 - ...spinach **sautéed with garlic over rice**
 - Chard stems and **leaves** are **delicious**
 - Collard greens and other **salty** leafy greens
- Conclusion:
 - Ong choi is a leafy green like spinach, chard, or collard greens



Meaning as distributional similarity

- Each word = a vector (or embedding)
 - Not just "word" or word_45.
 -
- Similar words are "nearby in space"

Intuition: why vectors?

Words as atomic units

No way to capture the similarity between words,
e.g. *amazing* and *fantastic*

Words as vectors

amazing has the vector $[35, 22, 17]$
fantastic has the vector $[34, 21, 14]$

- Similarity is captured by the distance between the vectors
- We can generalize from training set words to **similar but unseen** words!

Two kinds of word vectors

sparse

- Words are represented by (a simple function of) the **counts** of nearby words

dense

- A vector for a word is created by training a classifier to **predict** something about the word (e.g. whether it occurs nearby another word)

Sparse vectors

Two kinds of sparse vectors

Document vectors

- Used to represent documents

Word vectors

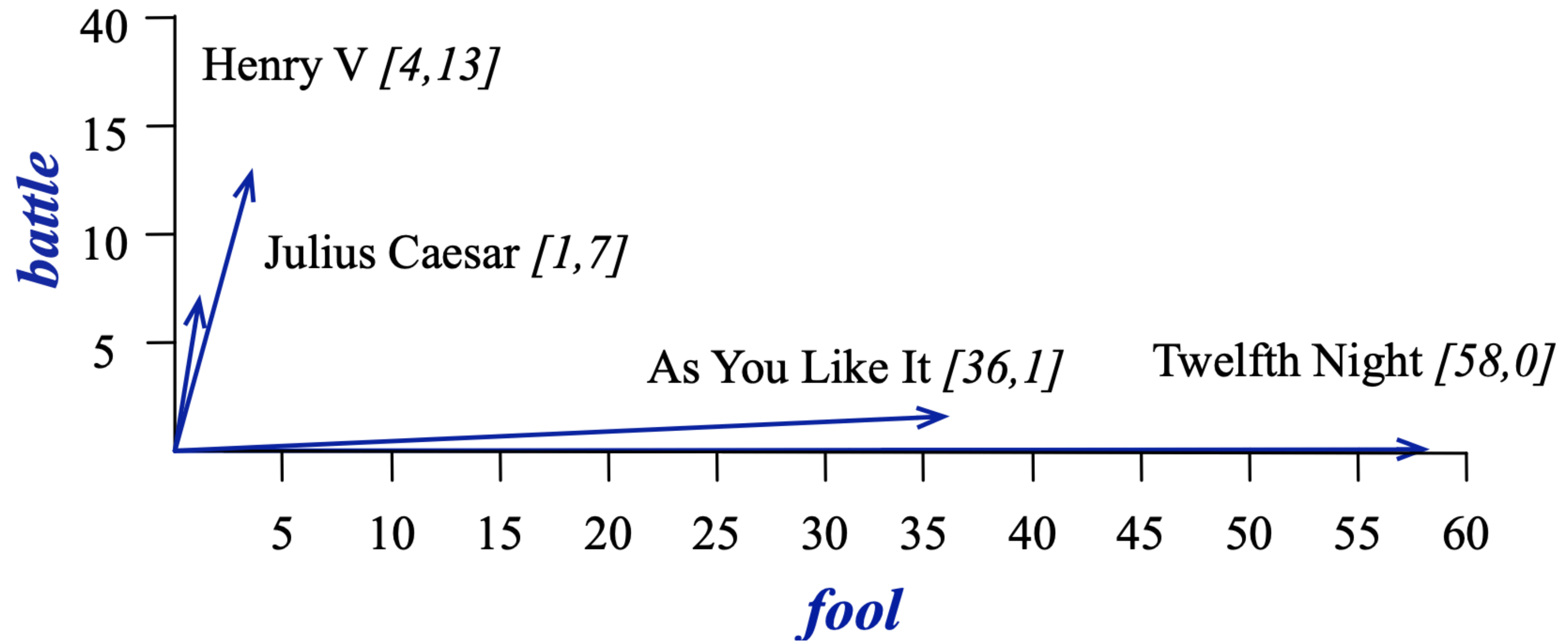
- Used to represent words

Term-document (or document-term) matrix

Each document is represented by a vector of words

	As You Like It	Twelfth Night	Julius Caesar	Henry V
battle	1	0	7	13
good	114	80	62	89
fool	36	58	1	4
wit	20	15	2	3

Visualizing document vectors



Words can be vectors too!!!

	As You Like It	Twelfth Night	Julius Caesar	Henry V
battle	1	0	7	13
good	114	80	62	89
fool	36	58	1	4
wit	20	15	2	3

battle is "the kind of word that occurs in Julius Caesar and Henry V"

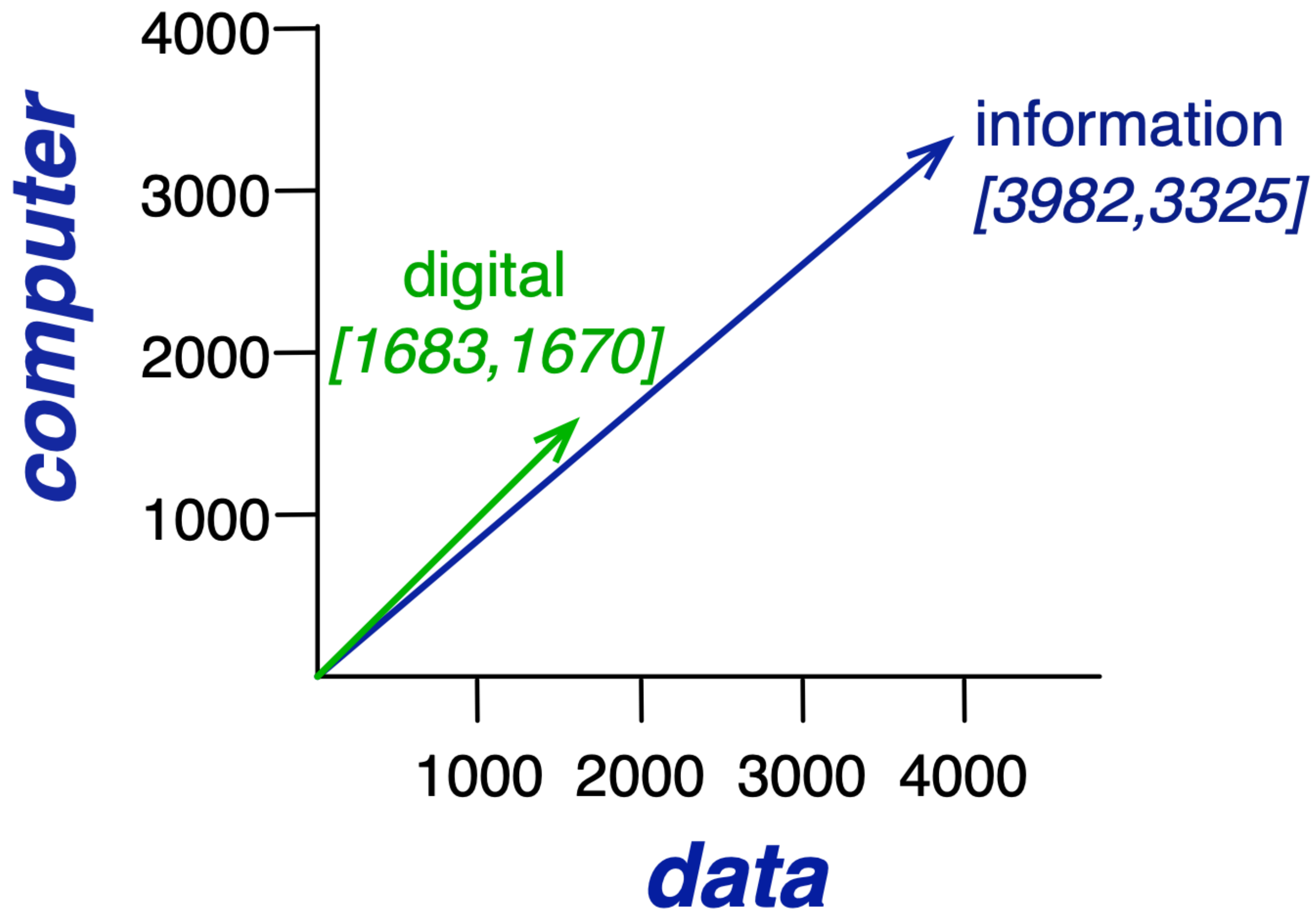
fool is "the kind of word that occurs in comedies, especially Twelfth Night"

More common: word-word matrix (or "term-context matrix")

Two **words** are similar in meaning if their context vectors are similar

is traditionally followed by **cherry** pie, a traditional dessert
often mixed, such as **strawberry** rhubarb pie. Apple pie
computer peripherals and personal **digital** assistants. These devices usually
a computer. This includes **information** available on the internet

	aardvark	...	computer	data	result	pie	sugar	...
cherry	0	...	2	8	9	442	25	...
strawberry	0	...	0	0	1	60	19	...
digital	0	...	1670	1683	85	5	4	...
information	0	...	3325	3982	378	5	13	...



Cosine Similarity

Dot product

- The dot product between two vectors is a scalar:

$$\text{dot product}(\mathbf{v}, \mathbf{w}) = \mathbf{v} \cdot \mathbf{w} = \sum_{i=1}^N v_i w_i = v_1 w_1 + v_2 w_2 + \dots + v_N w_N$$

- The dot product tends to be high when the two vectors have large values in the same dimensions
- Dot product can be a similarity metric between vectors

Problem with raw dot product

- Dot product favours long vectors
-
- Dot product is higher if a vector is longer (has higher values in many dimensions)
-
- Frequent words (*of, the, you*) have long vectors (since they occur many times with other words).
-
- So dot product overly favours frequent words

Alternative: cosine

$$\text{cosine}(\vec{v}, \vec{w}) = \frac{\vec{v} \cdot \vec{w}}{|\vec{v}| |\vec{w}|} = \frac{\sum_{i=1}^N v_i w_i}{\sqrt{\sum_{i=1}^N v_i^2} \sqrt{\sum_{i=1}^N w_i^2}}$$

Cosine examples

$$\cos(\vec{v}, \vec{w}) = \frac{\vec{v} \cdot \vec{w}}{|\vec{v}| |\vec{w}|} = \frac{\vec{v}}{|\vec{v}|} \cdot \frac{\vec{w}}{|\vec{w}|} = \frac{\sum_{i=1}^N v_i w_i}{\sqrt{\sum_{i=1}^N v_i^2} \sqrt{\sum_{i=1}^N w_i^2}}$$

	pie	data	computer
cherry	442	8	2
digital	5	1683	1670
information	5	3982	3325

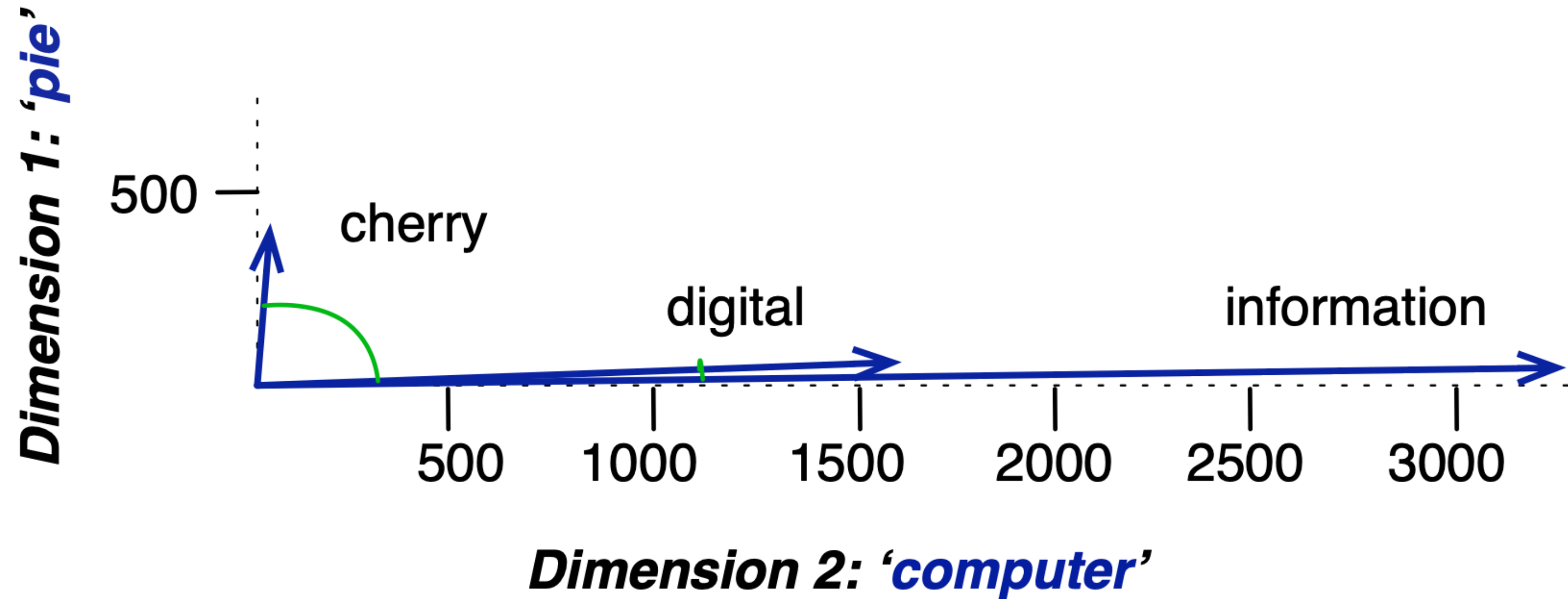
$$\cos(\text{cherry}, \text{information}) =$$

$$\frac{442 * 5 + 8 * 3982 + 2 * 3325}{\sqrt{442^2 + 8^2 + 2^2} \sqrt{5^2 + 3982^2 + 3325^2}} = .017$$

$$\cos(\text{digital}, \text{information}) =$$

$$\frac{5 * 5 + 1683 * 3982 + 1670 * 3325}{\sqrt{5^2 + 1683^2 + 1670^2} \sqrt{5^2 + 3982^2 + 3325^2}} = .996$$

Visualizing cosines (well, angles)



The problem with frequency counts

- Frequency is clearly useful; if *sugar* appears a lot near *apricot*, that's useful information.
- But overly frequent words like *the*, *it*, or *they* are not very informative about the context
- To overcome this, we weight the frequency counts in some way:
 - **TF-IDF** (term-document counts)
 - **PMI** (word co-occurrence counts)

TF-IDF

Term frequency (tf)

- $tft_{t,d} = \text{count}(t,d)$
- Instead of using raw count, we squash a bit:
- $tft_{t,d} = \log_{10}(\text{count}(t,d)+1)$

Document frequency (df)

- **df t** is the number of documents **t** occurs in.
- "*Romeo*" is very distinctive for one Shakespeare play:

	Collection Frequency	Document Frequency
Romeo	113	1
action	113	31

Inverse document frequency (idf)

$$\text{idf}_t = \log_{10} \left(\frac{N}{\text{df}_t} \right)$$

N is the total number of documents in the collection (in our example, 37)

Word	df	idf
Romeo	1	1.57
salad	2	1.27
Falstaff	4	0.967
forest	12	0.489
battle	21	0.246
wit	34	0.037
fool	36	0.012
good	37	0
sweet	37	0

Final tf-idf weighted value for a word

Raw counts:

	As You Like It	Twelfth Night	Julius Caesar	Henry V
battle	1	0	7	13
good	114	80	62	89
fool	36	58	1	4
wit	20	15	2	3

Tf-idf:

	As You Like It	Twelfth Night	Julius Caesar	Henry V
battle	0.074	0	0.22	0.28
good	0	0	0	0
fool	0.019	0.021	0.0036	0.0083
wit	0.049	0.044	0.018	0.022

PMI

Pointwise Mutual Information

Are two words co-occurring more than we would expect them to occur together by chance?

If two words are independent we expect them to occur with:

$$P(word_1)P(word_2)$$

We can then count how often they actually occur together:

$$P(word_1, word_2)$$

Pointwise Mutual Information

Are two words co-occurring more than we would expect them to occur together by chance?

$$PMI(word_1, word_2) = \log_2 \frac{P(word_1, word_2)}{P(word_1)P(word_2)}$$

Positive Pointwise Mutual Information

PMI ranges from $-\infty$ to $+\infty$

We replace negative PMI values by 0

Positive PMI (**PPMI**) between word 1 and word 2:

$$PPMI(word_1, word_2) = \max \left(\log_2 \frac{P(word_1, word_2)}{P(word_1)P(word_2)}, 0 \right)$$

Computing PPMI

Matrix F with W rows (words) and C columns (contexts)

f_{ij} is # of times w_i occurs in context c_j

$$p_{ij} = \frac{f_{ij}}{\sum_{i=1}^W \sum_{j=1}^C f_{ij}} \quad p_{i*} = \frac{\sum_{j=1}^C f_{ij}}{\sum_{i=1}^W \sum_{j=1}^C f_{ij}} \quad p_{*j} = \frac{\sum_{i=1}^W f_{ij}}{\sum_{i=1}^W \sum_{j=1}^C f_{ij}}$$

	computer	data	result	pie	sugar	count(w)
cherry	2	8	9	442	25	486
strawberry	0	0	1	60	19	80
digital	1670	1683	85	5	4	3447
information	3325	3982	378	5	13	7703
count(context)	4997	5673	473	512	61	11716

$$pmi_{ij} = \log_2 \frac{p_{ij}}{p_{i*} p_{*j}} \quad ppmi_{ij} = \begin{cases} pmi_{ij} & \text{if } pmi_{ij} > 0 \\ 0 & \text{otherwise} \end{cases}$$

	computer	data	result	pie	sugar	count(w)
cherry	2	8	9	442	25	486
strawberry	0	0	1	60	19	80
digital	1670	1683	85	5	4	3447
information	3325	3982	378	5	13	7703
count(context)	4997	5673	473	512	61	11716

$$p(w=\text{information}, c=\text{data}) = 3982/11716 = .3399$$

$$p(w=\text{information}) = 7703/11716 = .6575$$

$$p(c=\text{data}) = 5673/11716 = .4842$$

	p(w,context)					p(w)
	computer	data	result	pie	sugar	p(w)
cherry	0.0002	0.0007	0.0008	0.0377	0.0021	0.0415
strawberry	0.0000	0.0000	0.0001	0.0051	0.0016	0.0068
digital	0.1425	0.1436	0.0073	0.0004	0.0003	0.2942
information	0.2838	0.3399	0.0323	0.0004	0.0011	0.6575
p(context)	0.4265	0.4842	0.0404	0.0437	0.0052	

Resulting PPMI matrix (get log values and replaced negatives with 0)

	computer	data	result	pie	sugar
cherry	0	0	0	4.38	3.30
strawberry	0	0	0	4.10	5.51
digital	0.18	0.01	0	0	0
information	0.02	0.09	0.28	0	0

Weighting PMI

- PMI is biased toward infrequent events
 - Very rare words have very high PMI values
- How to mitigate this
 - Give rare words slightly higher probabilities

Dense vectors

Sparse versus dense vectors

- **sparse** vectors are
 - long (10s of thousands of dimensions)
 - most values are zero
- **dense** vectors are
 - short (length 50–1000)
 - most values are non-zero

In practice, **dense** vectors work better

Common methods for obtaining dense vectors

- Singular Value Decomposition (SVD)
 - A special case of this is called LSA – Latent Semantic Analysis
- “Neural Language Models”
 - **Static** word embeddings (**Word2vec**, GloVe)
 - **Dynamic** word embeddings (ELMo, BERT, GPT)

word2vec

Word2Vec: Skip-Gram Task

Word2vec provides a variety of options.
We'll do:

skip-gram with negative sampling (SGNS)

Word2vec

- Instead of **counting** how often two words occur together
 - Train a classifier on a binary **prediction** task:
 - Are the two words likely to appear together?
- We don't actually care about this task
 - But we want the word vectors that are learnt

Word2vec

- Instead of **counting** how often two words occur together
 - Train a classifier on a binary **prediction** task:
 - Are the two words likely to appear together?
- We don't actually care about this task
 - But we want the word vectors that are learnt

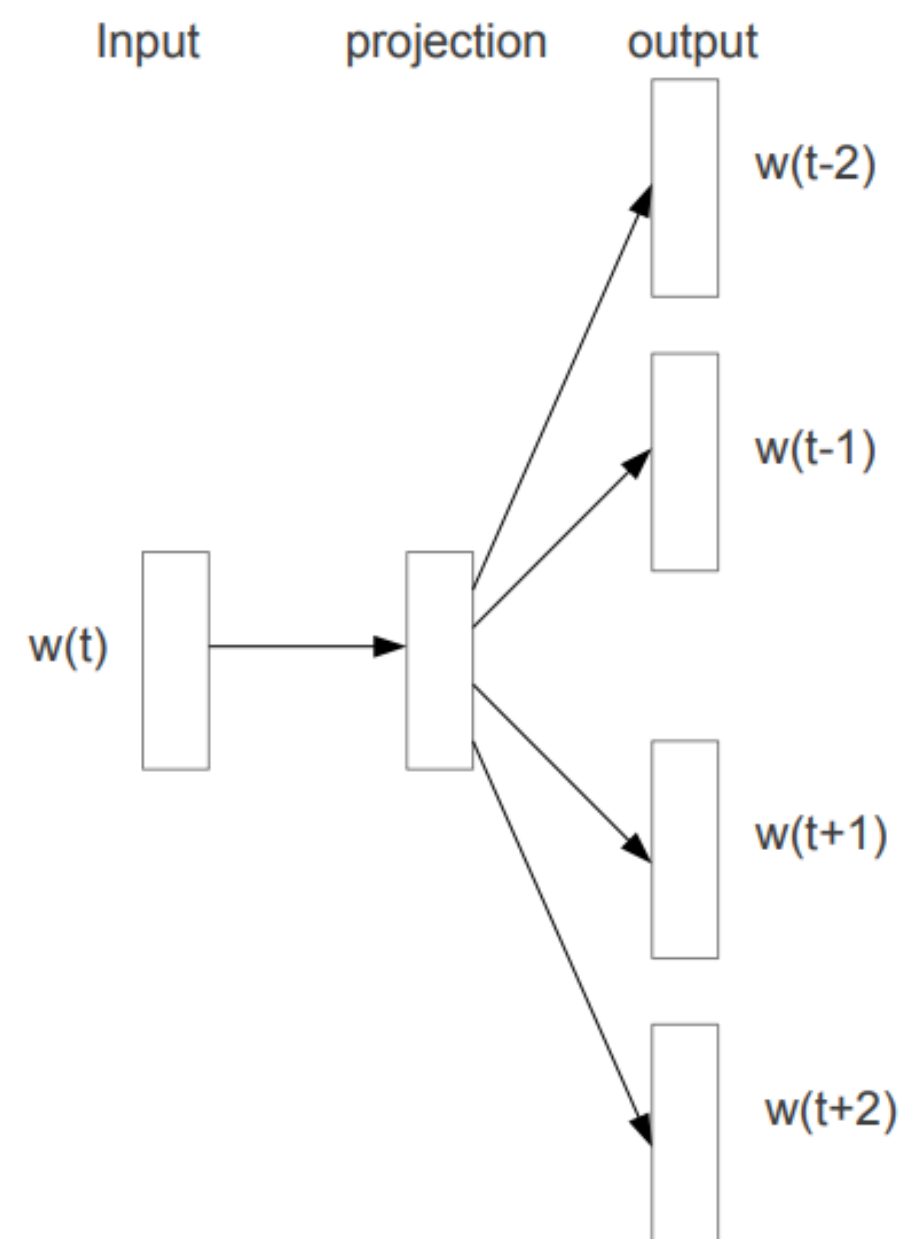


Figure 1: The Skip-gram model architecture. The training objective is to learn word vector representations that are good at predicting the nearby words.

Approach: predict if candidate word c is a "neighbour" of word w

1. Treat the target word w and a neighbouring context word c as **positive** examples
2. Randomly sample other words in the vocabulary to get **negative** examples
3. Use logistic regression to train a classifier to distinguish those two cases
4. Target and context words are represented as word vectors
5. During training, the classifier adjusts the word vectors (the word vectors are the weights)

What counts as “neighbouring”?

Assume a ± 2 word window, given training sentence:

...lemon, a [tablespoon of apricot jam, a] pinch...

c1 c2 [target] c3 c4

Skip-Gram Classifier

Goal

train a classifier that is given a candidate
(**w**ord, **c**ontext) pair:

(apricot, tablespoon)
(apricot, aardvark)

...

and assigns each pair a probability:
 $P(+|w, c)$

How do we calculate $p(+|w,c)$?

- Remember w and c are word **vectors**
- If two words co-occur regularly, their vectors should be similar
- Use $w \cdot c$ (**dot product**)
- Convert to a probability using the sigmoid function

How do we calculate $p(+|w,c)$?

$$P(+|w,c) = \sigma(c \cdot w) = \frac{1}{1 + \exp(-c \cdot w)}$$

$$\begin{aligned} P(-|w,c) &= 1 - P(+|w,c) \\ &= \sigma(-c \cdot w) = \frac{1}{1 + \exp(c \cdot w)} \end{aligned}$$

How do we calculate $p(+|w,c)$?

- For each target word, we will have more than one context word.
- We'll assume independence and just multiply them:

$$P(+|w, c_{1:L}) = \prod_{i=1}^L \sigma(c_i \cdot w)$$
$$\log P(+|w, c_{1:L}) = \sum_{i=1}^L \log \sigma(c_i \cdot w)$$

Skip-gram classifier: summary

A probabilistic classifier that, given

- a target word w
- its context window of L words $c_1:L$,
assigns a probability that w occurs in this window.

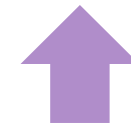
To compute this, we just need vectors for all the words.

Learning word2vec embeddings

Training data

...lemon, a [tablespoon of apricot jam, a] pinch...

c1 c2 [target] c3 c4



positive examples +

t	c
apricot	tablespoon
apricot	of
apricot	jam
apricot	a

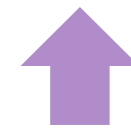
Training data

...lemon, a [tablespoon of apricot jam, a] pinch...

c1 c2 [target] c3 c4

positive examples +

t	c
apricot	tablespoon
apricot	of
apricot	jam
apricot	a



For each positive example
we'll grab k negative
examples, sampling by
frequency

Training data

...lemon, a [tablespoon of apricot jam, a] pinch...

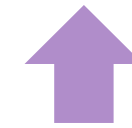
c1 c2 [target] c3 c4

positive examples +

t	c
apricot	tablespoon
apricot	of
apricot	jam
apricot	a

negative examples -

t	c	t	c
apricot	aardvark	apricot	seven
apricot	my	apricot	forever
apricot	where	apricot	dear
apricot	coaxial	apricot	if



Training

Given the set of positive and negative training instances, and an initial set of word vectors , the goal of learning is to adjust those word vectors such that we:

- **Maximize** the similarity of the target word, context word pairs (w, c_{pos}) drawn from the positive data
- **Minimize** the similarity of the (w, c_{neg}) pairs drawn from the negative data.

Loss function for one w with c_{pos} , c_{neg_1} ... c_{neg_k}

Maximize the **dot product** of the word with the actual context words, and minimize the **dot products** of the word with the k negative sampled non-neighbour words.

$$\begin{aligned} L_{CE} &= -\log \left[P(+|w, c_{pos}) \prod_{i=1}^k P(-|w, c_{neg_i}) \right] \\ &= - \left[\log P(+|w, c_{pos}) + \sum_{i=1}^k \log P(-|w, c_{neg_i}) \right] \\ &= - \left[\log P(+|w, c_{pos}) + \sum_{i=1}^k \log (1 - P(+|w, c_{neg_i})) \right] \\ &= - \left[\log \sigma(c_{pos} \cdot w) + \sum_{i=1}^k \log \sigma(-c_{neg_i} \cdot w) \right] \end{aligned}$$

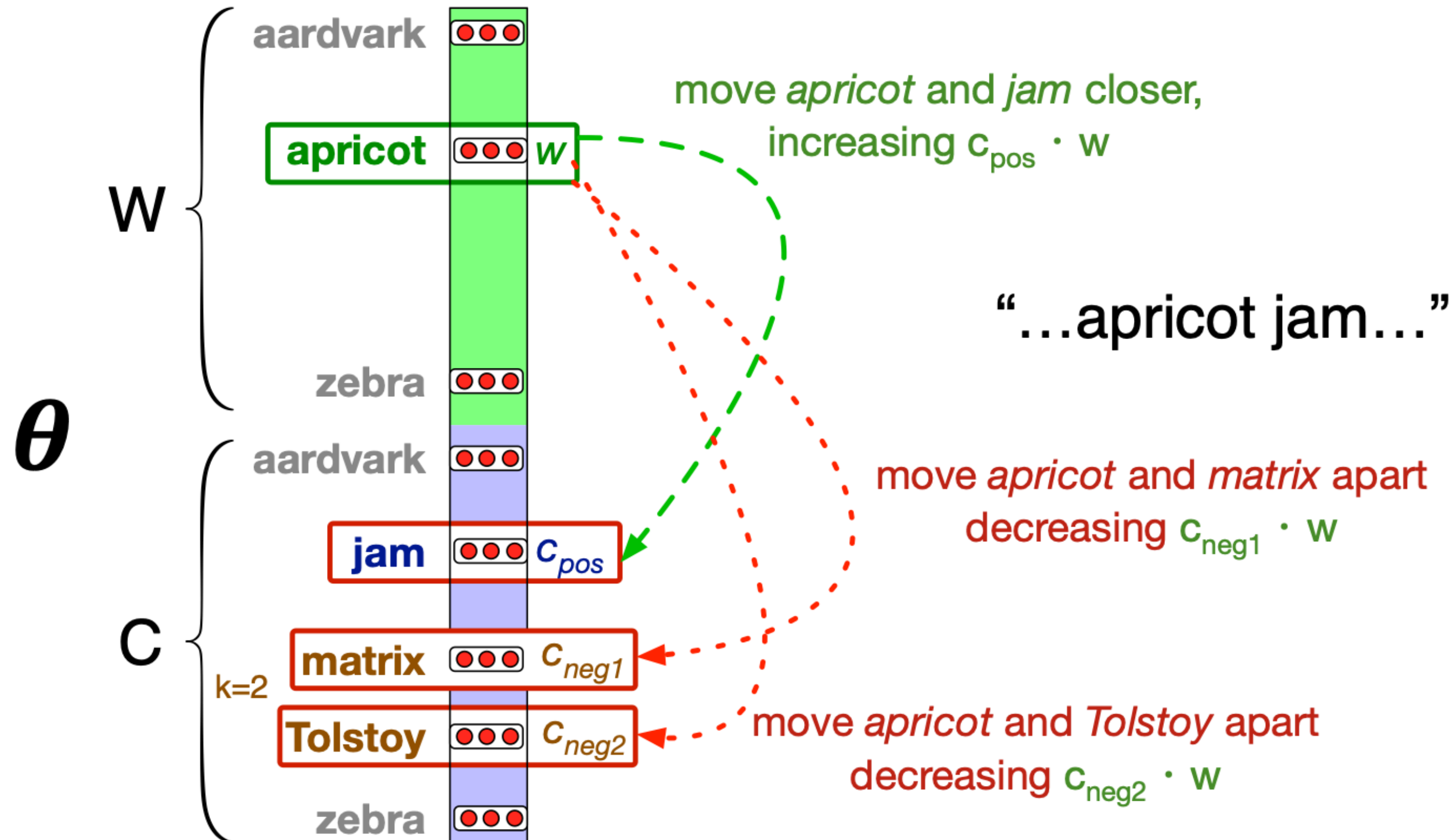
How to learn?

Stochastic gradient descent!

We'll adjust the word weights (i.e. the word vectors) to

- make the positive pairs more likely
- and the negative pairs less likely,
over the entire training set.

Intuition of one step of gradient descent



The derivatives of the loss function

$$L_{CE} = - \left[\log \sigma(c_{pos} \cdot w) + \sum_{i=1}^k \log \sigma(-c_{neg_i} \cdot w) \right]$$

$$\frac{\partial L_{CE}}{\partial c_{pos}} = [\sigma(c_{pos} \cdot w) - 1]w$$

$$\frac{\partial L_{CE}}{\partial c_{neg}} = [\sigma(c_{neg} \cdot w)]w$$

$$\frac{\partial L_{CE}}{\partial w} = [\sigma(c_{pos} \cdot w) - 1]c_{pos} + \sum_{i=1}^k [\sigma(c_{neg_i} \cdot w)]c_{neg_i}$$

Updating the word vectors

$$c_{pos}^{t+1} = c_{pos}^t - \eta [\sigma(c_{pos}^t \cdot w) - 1] w$$

$$c_{neg}^{t+1} = c_{neg}^t - \eta [\sigma(c_{neg}^t \cdot w)] w$$

$$w^{t+1} = w^t - \eta [\sigma(c_{pos} \cdot w^t) - 1] c_{pos} + \sum_{i=1}^k [\sigma(c_{neg_i} \cdot w^t)] c_{neg_i}$$

Two sets of vectors

SGNS learns *two* sets of vectors:

1. Target embedding matrix W
2. Context embedding matrix C

It's common to just add them together,
representing word i as the vector $w_i + c_i$

Summary: How to learn word2vec (skip-gram) vectors

Start with $2V$ random d -dimensional vectors as initial vectors

Train a classifier based on vector similarity

1. Take a corpus and take pairs of words that co-occur as positive examples.
2. Take pairs of words that don't co-occur as negative examples.
3. Train the classifier to distinguish these by iteratively adjusting all the vectors.
4. Throw away the classifier code and keep the vectors.

Properties of word2vec embeddings

The effect of context window size

Large windows ($C = +/- 5$): nearest words are related words in same semantic field

- *Hogwarts* nearest neighbours are Harry Potter world:

Dumbledore, Half-blood, Malfoy

Small windows ($C = +/- 2$): nearest words are similar nouns, words in same taxonomy

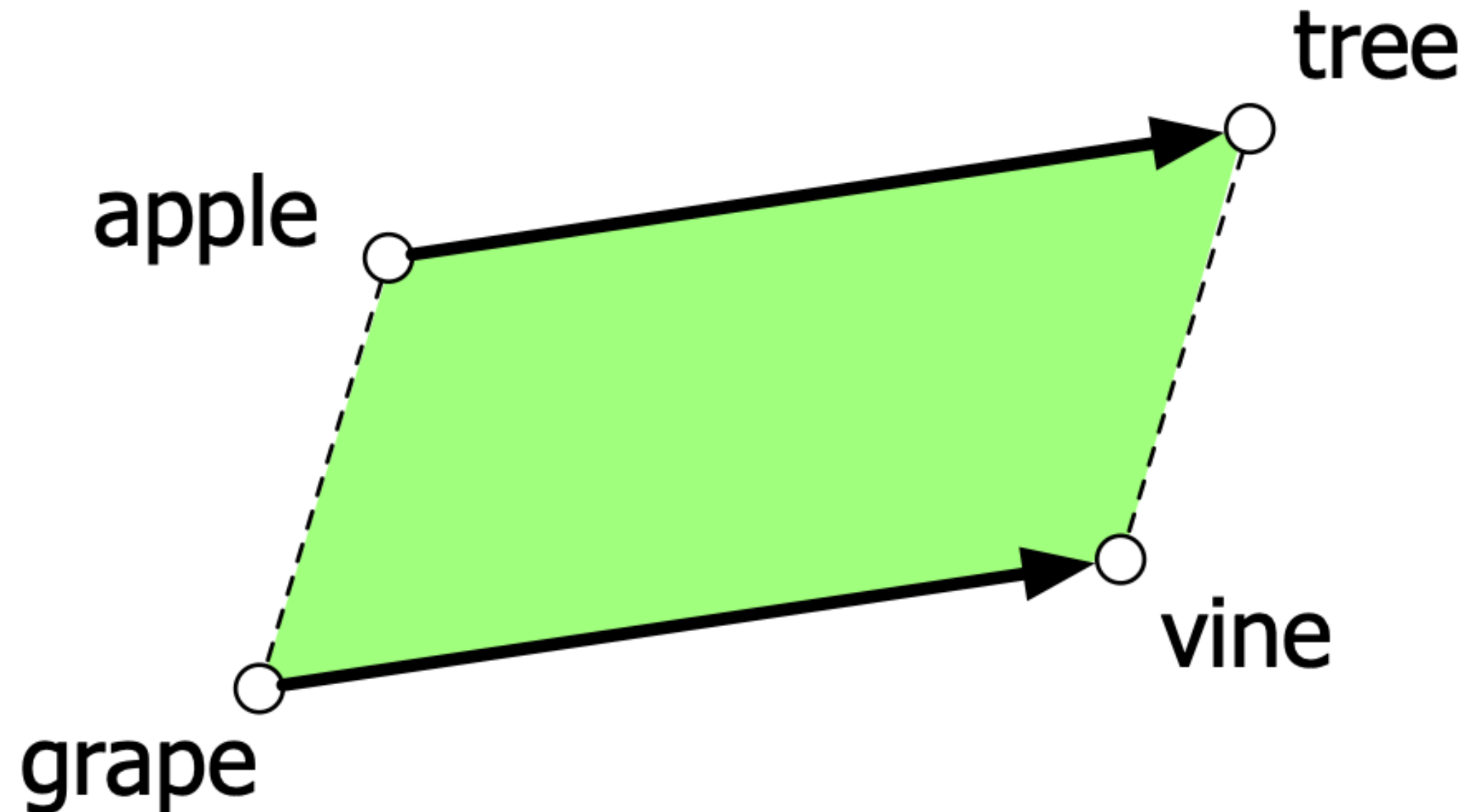
Hogwarts nearest neighbours are other fictional schools

- *Sunnydale, Evernight, Blandings*

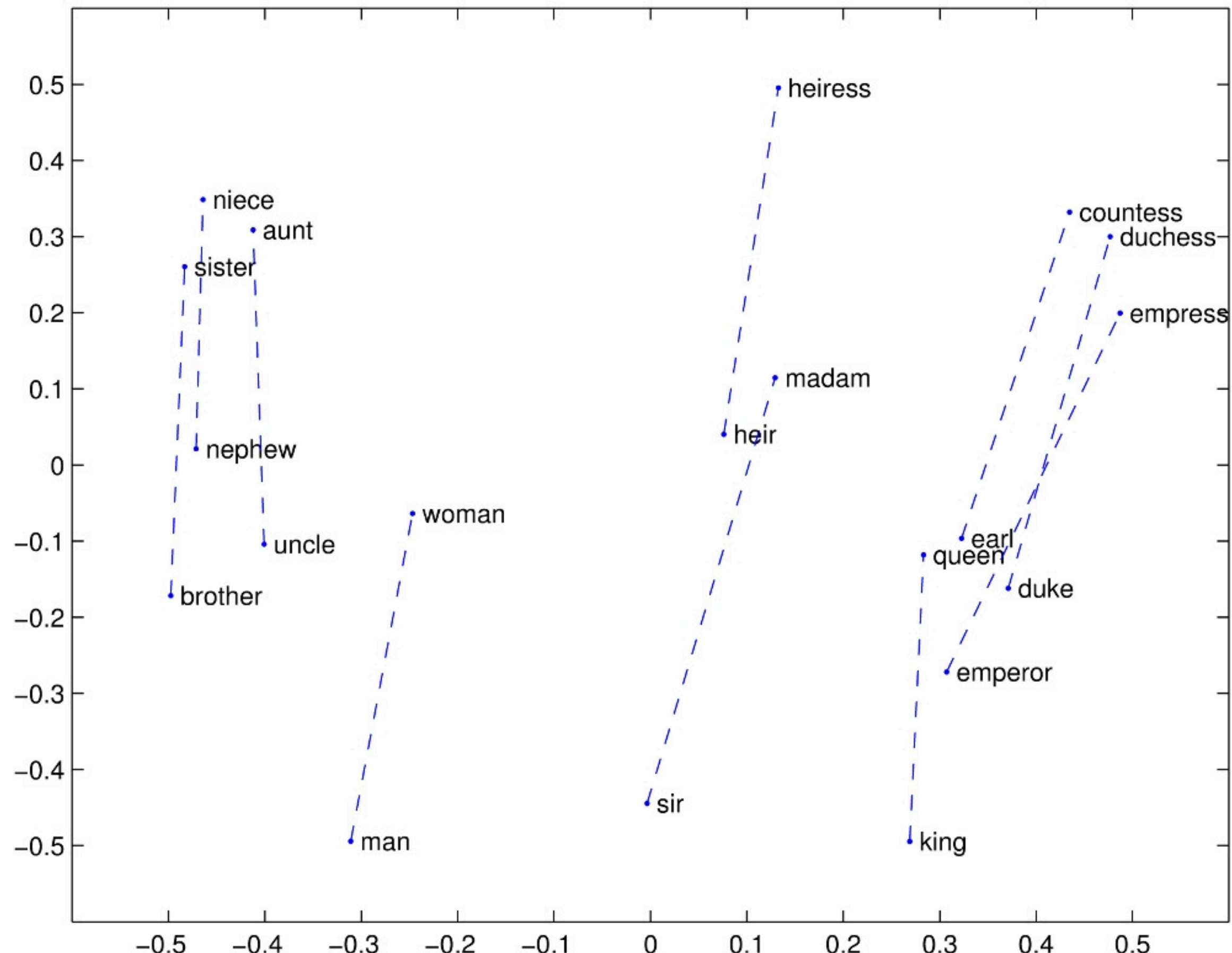
Analogical relations

To solve: *"apple is to tree as grape is to _____"*

Add *apple – tree* to grape to get *vine*



Structure in GloVe Embedding space

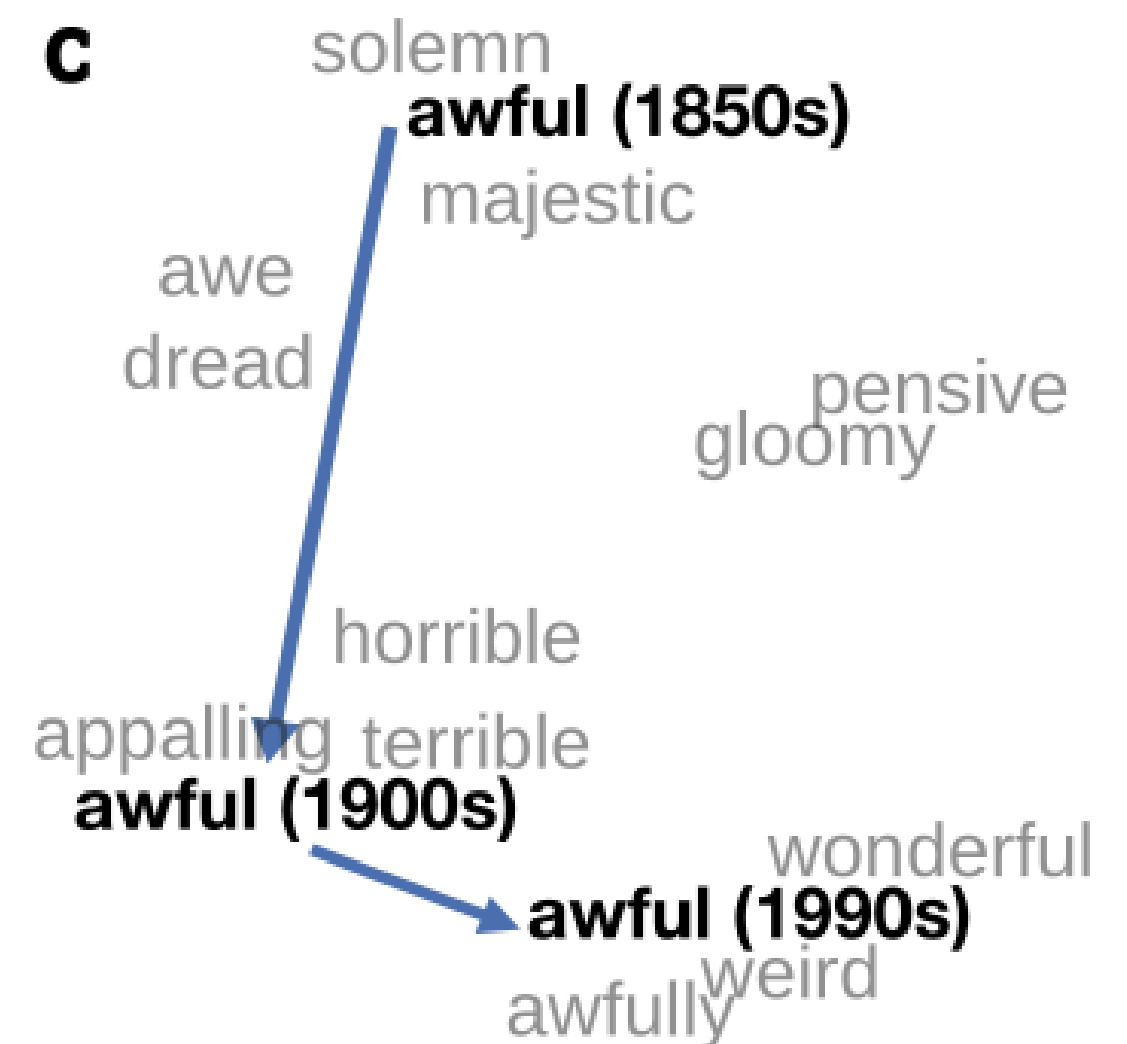
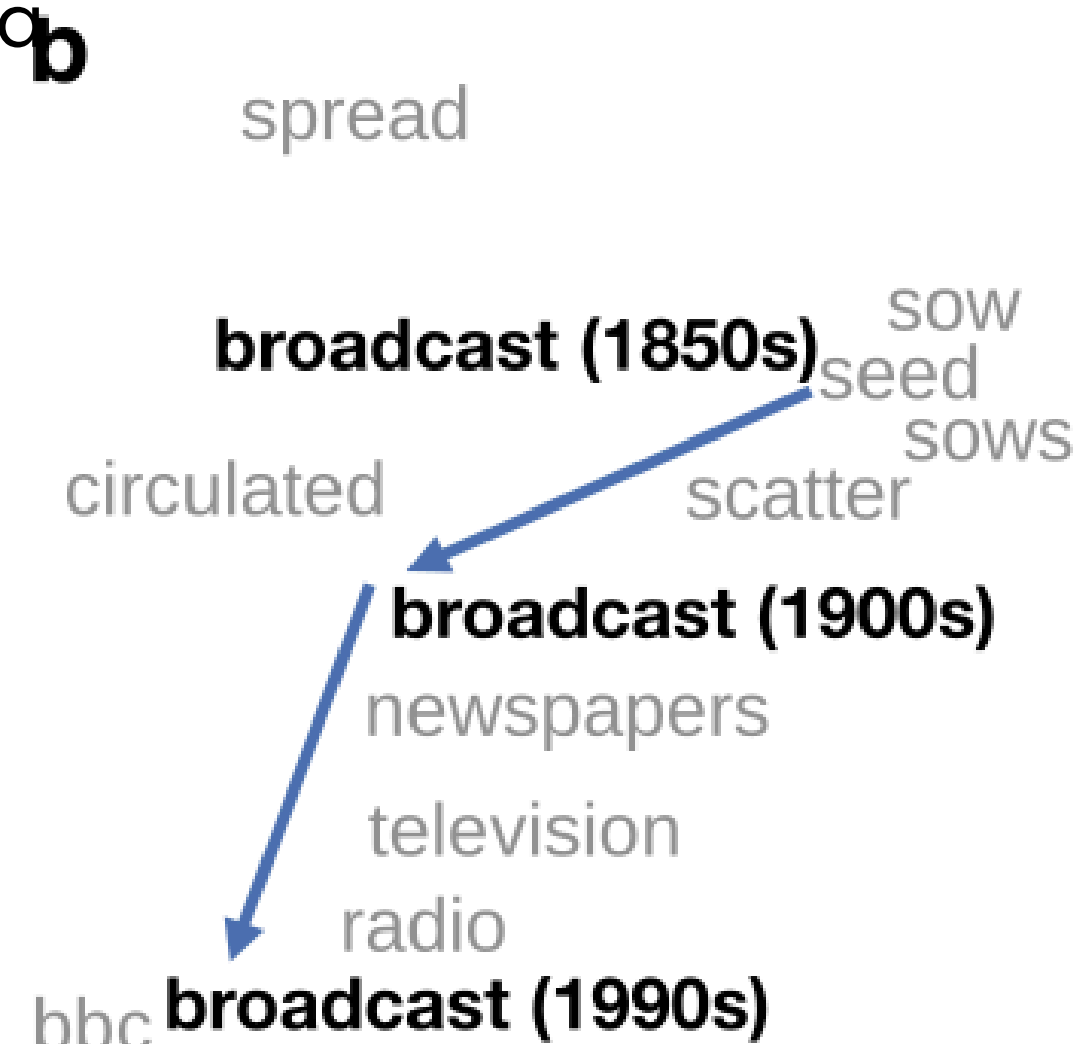
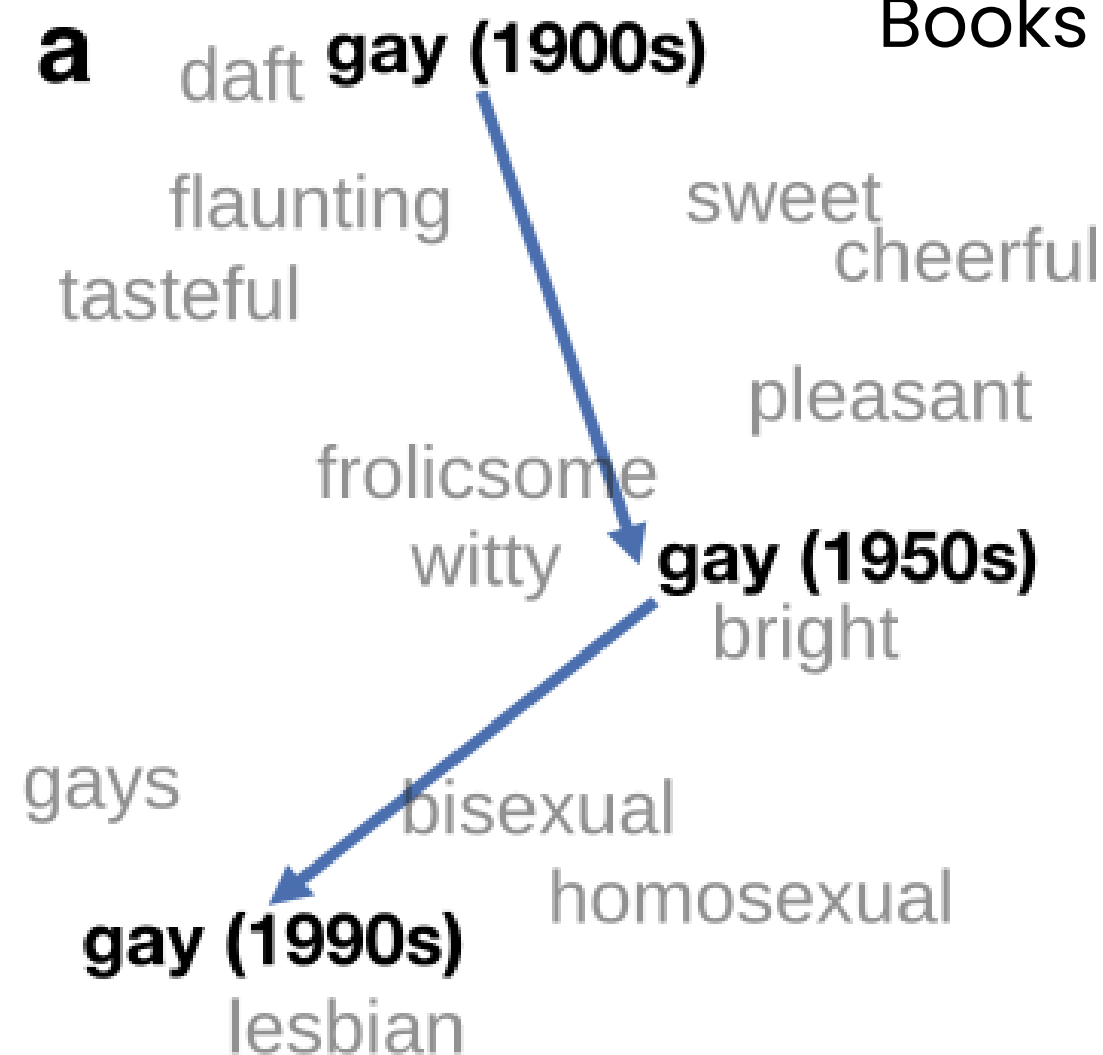


Embeddings for historical linguistics

Train embeddings on different decades of historical text to see meanings shift

~30 million books, 1850–1990, Google

Books data



Embeddings reflect cultural bias!

- Ask “Paris : France :: Tokyo : x”
 - x = Japan
- Ask “father : doctor :: mother : x”
 - x = nurse
- Ask “man : computer programmer :: woman : x”
 - x = homemaker